

Part A — Conceptual Foundation

1. Short Notes

A. What is Data Analysis?

Data Analysis is the process of collecting, cleaning, transforming, exploring, and interpreting data to find useful information, patterns, and insights.

Main steps of Data Analysis:

1. Data Collection – Gather data from sources such as CSV, databases, APIs, surveys, etc.
2. Data Cleaning – Handle missing values, duplicates, incorrect data, and outliers.
3. Data Exploration – Understand the structure and characteristics of the dataset.
4. Data Transformation – Convert data into a suitable format using encoding, scaling, normalization, etc.
5. Data Visualization – Represent information using graphs and charts.
6. Interpretation – Identify patterns and relationships.
7. Decision Making – Use the findings to support business or practical decisions.

Example:

A company can analyze customer purchase data to identify its most popular products and understand customer buying patterns.

B. How to Plan a Data Science Project

Planning a Data Science project means defining the problem and creating a systematic process to solve it using data.

Steps:

1. Define the Problem
Clearly identify what problem needs to be solved.
2. Set Objectives
Define what the project should achieve.
3. Collect Data
Gather relevant data from databases, files, APIs, websites, or other sources.
4. Understand the Data
Examine columns, data types, distributions, and relationships.

5. Clean and Preprocess Data
Handle missing values, duplicates, outliers, encoding, and scaling.
6. Exploratory Data Analysis (EDA)
Analyze patterns and relationships using statistics and visualization.
7. Feature Engineering
Create useful features from existing data.
8. Model Building
Select and train suitable Machine Learning algorithms if required.
9. Model Evaluation
Evaluate the model using appropriate performance metrics.
10. Deployment and Monitoring
Deploy the solution and monitor its performance over time.

C. How to Frame a Machine Learning Problem

Framing a Machine Learning problem means converting a real-world problem into a well-defined ML problem that can be solved using data.

Steps:

1. Identify the Business/Real-World Problem
Understand what needs to be predicted or classified.
2. Define the Objective
Clearly state what the ML model should accomplish.
3. Identify the Target Variable
Determine what the model needs to predict.
4. Identify Features
Select the input variables that can help predict the target.
5. Determine the Problem Type
 - Classification → Predict categories.
 - Regression → Predict continuous numerical values.
 - Clustering → Group similar observations.
 - Time Series → Predict future values based on time-based data.
6. Collect Relevant Data
Obtain sufficient and reliable data.

7. Define Evaluation Metrics

Select suitable metrics such as Accuracy, Precision, Recall, F1-score, MAE, or RMSE.

8. Check Constraints

Consider computational resources, time, data availability, and interpretability.

Example:

If a bank wants to predict whether a customer will default on a loan:

- Target → default_flag
- Features → income, credit score, loan amount, repayment history, etc.
- Problem type → Binary Classification

2. Tensors

What is a Tensor?

A tensor is a multi-dimensional array used to represent and process numerical data.

A tensor can have different numbers of dimensions:

Tensor	Dimension	Example
Scalar	0D	5
Vector	1D	[1, 2, 3]
Matrix	2D	[[1,2], [3,4]]
3D Tensor	3D	Collection of matrices
Higher-order Tensor	4D+	Multiple 3D arrays

Tensor Example

Scalar:

5

Vector:

[1 2 3]

Matrix:

```
[1 2  
3 4]
```

3D Tensor:

```
[  
  [[1,2],  
   [3,4]],  
  
  [[5,6],  
   [7,8]]  
]
```

Tensors using NumPy

NumPy provides the ndarray object for working with multi-dimensional arrays.

```
import numpy as np
```

```
tensor = np.array(  
    [[1, 2], [3, 4]],  
    [[5, 6], [7, 8]]  
)
```

```
print(tensor)
```

Check dimensions

```
print(tensor.ndim)
```

Output:

3

So this is a **3-dimensional tensor**.

Check shape

```
print(tensor.shape)
```

Output:

(2, 2, 2)

This means:

- 2 blocks
- 2 rows in each block
- 2 columns in each row

Important NumPy Tensor Properties

1. ndim

Returns the number of dimensions.

```
tensor.ndim
```

2. shape

Returns the size of each dimension.

```
tensor.shape
```

3. size

Returns the total number of elements.

```
tensor.size
```

For the above tensor:

8

4. dtype

Returns the data type of elements.

```
tensor.dtype
```

Tensor Operations using NumPy

Addition

```
a = np.array([[1, 2], [3, 4]])
```

```
b = np.array([[5, 6], [7, 8]])
```

```
print(a + b)
```

Output:

```
[[ 6  8]
```

```
 [10 12]]
```

Multiplication

Element-wise multiplication:

```
print(a * b)
```

Matrix Multiplication

```
print(a @ b)
```

Reshaping

```
a = np.array([1, 2, 3, 4, 5, 6])
```

```
b = a.reshape(2, 3)
```

```
print(b)
```

Output:

```
[[1 2 3]
```

```
 [4 5 6]]
```

Indexing

```
print(tensor[0, 1, 1])
```

This accesses an individual element from the 3D tensor.